

Buffer Engine

[1. BUFFER Engine - Kompakte System-Referenz für KI-Modelle](#)

[2. Operationen \(DSL op\)](#)

[2.1. EXEC \(Variablendefinition\)](#)

[2.2. ASSERT \(Validierung\)](#)

[Unterstützte Validierungs-Aktionen \(action\):](#)

[3. Referenz der BUFFER-Parameter](#)

[Praxis-Tutorial: BUFFER Engine Demonstration](#)

[Vollständiges YAML-Szenario](#)

[Log-Output der Engine](#)

[Troubleshooting & Best Practices](#)

[Troubleshooting-Checkliste](#)

[Performance-Hinweise](#)

Die BUFFER-Engine ist das zentrale Modul für die Verwaltung lokaler Variablen und die Durchführung einfacher textbasierter Validierungen innerhalb des AGATE-Test-Frameworks. Sie ermöglicht es, Daten während eines Testlaufs flexibel im Speicher abzugeben, zu manipulieren und gegen definierte Erwartungswerte zu prüfen.

1. BUFFER Engine - Kompakte System-Referenz für KI-Modelle

1 | YAML

```
1 # AGATE BUFFER-Engine Syntax Referenz
2 # Nutze dies für Testfall-Generierung und Analyse
3
4 - type: BUFFER
5   op: EXEC
6   name: "... "          # Variablenname (Bezeichner)
7   value: "... "         # Statischer Wert oder dynamisch: "{B[var]}-suf
8   condition: "... "     # Optional: '{B[var]}' == 'val' (Einfache Anfüh
9
10 - type: BUFFER
11   op: ASSERT
12   name: "... "          # Name der zu prüfenden Variable
13   action: <EQUALS|NOT_EQUALS|CONTAINS|IS_NULL|IS_NOT_NULL|IS_EMPTY|IS_N
14   expected: "... "      # Erwarteter Wert (nur bei EQUALS, NOT_EQUALS,
15   condition: "... "     # Optional
16
```

2. Operationen (DSL op)

Die BUFFER-Engine basiert auf einem unkomplizierten In-Memory-Speicherprinzip. Variablen werden im Arbeitsspeicher unter einem eindeutigen `name` abgelegt, auf den in nachfolgenden Schritten über das `{B[name]}`-Tag zugegriffen werden kann.

2.1. EXEC (Variablendefinition)

Dient zum Setzen oder Überschreiben eines Wertes im Kontext des aktuellen Testfalls. Der Wert kann statisch sein oder dynamische Platzhalter enthalten, die vor der Speicherung aufgelöst

werden.

Parameter	Beschreibung	Erforderlich
name	Der eindeutige Bezeichner der Variable.	Ja
value	Der zu speichernde Inhalt. Unterstützt die Verschachtelung anderer Variablen.	Ja
condition	<i>Optional:</i> Bedingung für die Ausführung. Der Schritt wird übersprungen (<code>SKIPPED</code>), wenn die Bedingung <code>false</code> ergibt.	Nein

```
1 - type: BUFFER
2   op: EXEC
3   name: "app_env"
4   value: "PROD-Server-01"
```

2.2. ASSERT (Validierung)

Prüft den Inhalt einer zuvor gespeicherten Variable gegen ein definiertes Kriterium. Schlägt die Prüfung fehl, wird die Ausführung sofort mit einer detaillierten Fehlermeldung abgebrochen (Fail-Fast).

Parameter	Beschreibung	Erforderlich
name	Name der zu validierenden Variable.	Ja
action	Die Vergleichsoperation (siehe unterstützte Validierungs-Aktionen).	Ja

expected	Der erwartete Vergleichswert (wird bei <code>IS_...</code> Aktionen ignoriert).	Nur bei EQUALS, NOT_EQUALS, CONTAINS
condition	<i>Optional:</i> Bedingung für die Ausführung.	Nein

Unterstützte Validierungs-Aktionen (`action`):

Aktion	Beschreibung	Erwartet expected?
EQUALS	Prüft auf exakte Übereinstimmung der Zeichenketten.	Ja
NOT_EQUALS	Prüft, ob der Inhalt vom Erwartungswert abweicht.	Ja
CONTAINS	Prüft, ob der Erwartungswert als Teilstring im Buffer existiert.	Ja
IS_NULL	True, wenn die Variable nicht definiert oder explizit <code>null</code> ist.	Nein
IS_NOT_NULL	True, wenn die Variable existiert und einen Wert besitzt.	Nein
IS_EMPTY	True, wenn die Variable ein leerer String <code>""</code> (oder <code>null</code>) ist.	Nein
IS_NOT_EMPTY	True, wenn die Variable mindestens ein Zeichen enthält.	Nein

3. Referenz der BUFFER-Parameter

Hier sind alle Parameter für die BUFFER-Schritte im Überblick:

Schritt	Parameter	Beschreibung
EXEC	name	Der Name der Zielvariable im Speicher.
	value	Der zu speichernde Text-Inhalt (statisch oder dynamisch mit <code>{B[...]}</code>).
	condition	<i>Optional:</i> Bedingung für die Ausführung.
ASSERT	name	Der Name der zu prüfenden Variable.
	action	Die zugrundeliegende Prüf-Logik (z. B. <code>EQUALS</code> , <code>CONTAINS</code> , <code>IS_EMPTY</code>).
	expected	Der erwartete Wert für den Textvergleich (unterstützt Platzhalter).
	condition	<i>Optional:</i> Bedingung für die Ausführung.

Praxis-Tutorial: BUFFER Engine Demonstration

Dieses Beispiel deckt die Erstellung, die dynamische Verknüpfung komplexer Strings sowie die anschließende Validierung ab.

Vollständiges YAML-Szenario

1 | YAML

```
1 testCases:
2   - id: "TC_BUF_DEMO"
3     description: "Variablenspeicherung, String-Konstruktion und Prüfung"
```

```

4     variables:
5         local_prefix: "PRE-PROD"
6         app_name: "Payment-Processing"
7     steps:
8         # 1. Einfachen Wert in den Buffer schreiben
9         - type: BUFFER
10            op: EXEC
11            name: "application_env"
12            value: "Customer-Service-PROD"
13
14        # 2. Validierung des einfachen Wertes
15        - type: BUFFER
16            op: ASSERT
17            name: "application_env"
18            action: EQUALS
19            expected: "Customer-Service-PROD"
20
21        # 3. Dynamische Konstruktion eines neuen Strings aus bestehenden
22        - type: BUFFER
23            op: EXEC
24            name: "full_system_string"
25            value: "{B[local_prefix]}-{B[app_name]}-v2"
26
27        # 4. Prüfung der neu erstellten Variable auf Inhalt
28        - type: BUFFER
29            op: ASSERT
30            name: "full_system_string"
31            action: IS_NOT_EMPTY
32
33        # 5. Bedingte Ausführung basierend auf dem Buffer-Inhalt
34        - type: BUFFER
35            condition: "'{B[application_env]}' == 'Customer-Service-PROD'"
36            op: EXEC
37            name: "prod_flag"
38            value: "active"
39

```

Log-Output der Engine

Die BUFFER-Engine nutzt ein unifiziertes Log-Format mit standardisierten Trennstrichen (|).

Im `isVerbose` -Modus wird folgendes dokumentiert:

```

1     >>> BUFFER      : [application_env] | Store value [Customer-Service-
2     >>> ASSERT      : [application_env] | EQUALS "Customer-Service-PROD"
3     >>> RESULT      : SUCCESS | Value is "Customer-Service-PROD"
4     >>> BUFFER      : [full_system_string] | Store value [PRE-PROD-Payme
5     >>> ASSERT      : [full_system_string] | IS_NOT_EMPTY
6     >>> RESULT      : SUCCESS | Value is "PRE-PROD-Payment-Processing-v2

```

Troubleshooting & Best Practices

Wenn Variablen nicht aufgelöst werden oder Validierungen fehlschlagen, prüfe folgende Punkte:

Troubleshooting-Checkliste

- **Variablen-Auflösung schlägt fehl (Platzhalter bleibt als `{B[...]}` stehen):**
 - Überprüfe die Schreibweise des Variablennamens exakt. Die Keys im Speicher sind **Case-Sensitive** (Groß-/Kleinschreibung beachten).

- Stelle sicher, dass der `EXEC` -Schritt, welcher die Variable befüllt, zeitlich vor dem Schritt aufgerufen wird, der sie konsumiert.
- **Schritt wird unerwartet übersprungen (SKIPPED):**
 - Überprüfe die Syntax im `condition` -Feld. Da Werte direkt evaluiert werden, müssen Strings zwingend in einfachen Anführungszeichen stehen, z. B.: `condition: '{B[prod_flag]}' == 'active'`.
- **Unerwarteter Abbruch bei unbekannten Aktionen:**
 - Wenn im `ASSERT` -Schritt ein Tippfehler bei der `action` unterläuft (z. B. `EQUAL` statt `EQUALS`), bricht die Engine mit einer `RuntimeException: Unknown assertion type: "..."` ab, noch bevor der Vergleich stattfindet.

Performance-Hinweise

- **Arbeitsspeicher-Schonung:** Da alle Variablen direkt im RAM des ausgeführten Testcases (`tc.addVariable`) und im `ExecutionContext` gehalten werden, sollten keine extrem großen Text-Blöcke (z.B. komplette Log-Dateien von mehreren Megabytes) dauerhaft in BUFFER-Variablen überführt werden, wenn diese nicht zwingend für nachfolgende Schritte benötigt werden.